

Dipartimento di Ingegneria e Scienza dell'Informazione

Progetto:

IoSonoTrento

Titolo del documento:

Analisi e Progettazione

Doc. Name	<i>D3- IoSonoTrentoAnalisiProgettazione</i>	Doc. Number	D3 V1.0
Description	Documento di design		

Corso: Ingegneria del Software
Anno Accademico: 2025/2026

Indice

Scopo del documento	3
1 Analisi dei Componenti	3
1.1 Definizione dei componenti	3
CMP1: Gestione autenticazione	3
CMP2: Gestione database	4
CMP3: Gestione consultazioni	4
CMP4: Gestione iniziative	5
CMP5: Gestione profilo cittadino	6
CMP6: Gestione statistiche	6
CMP7: Scheduler consultazioni	7
CMP8: Sistema di notifiche	7
1.2 Diagramma dei componenti	8
2 Diagramma delle classi	8
1. Cittadino	8
2. Operatore	10
3. Consultazione	11
4. Domanda	12
5. Iniziativa	12
6. RispostaConsultazione	13
7. VotoIniziativa	15
8. Categoria Iniziativa	16
9. Notifica	16
2.1 Diagramma delle classi complessivo	17
3 Dal class diagram alle API	17

Scopo del documento

Il presente documento riporta il design del progetto **IoSonoTrento** piattaforma di partecipazione civica per il Comune di Trento attraverso l'utilizzo di un diagramma dei componenti e di un diagramma delle classi. Partendo dall'analisi dei requisiti e dagli use-case individuati nelle fasi precedenti, vengono qui formalizzati i componenti architetturali del sistema, le loro interfacce, e la struttura delle classi del dominio, con lo scopo di guidare l'implementazione backend e frontend dell'applicazione.

1. Analisi dei Componenti

1.1. Definizione dei componenti

In questa sezione vengono descritti i componenti con le apposite interfacce previste per il software **IoSonoTrento**.

CMP1: Gestione autenticazione

Descrizione:

Il componente si occupa dell'autenticazione degli utenti del sistema, che si divide in due percorsi distinti in base al ruolo. I *Cittadini* accedono esclusivamente tramite Google OAuth 2.0 (Single Sign-On): il sistema li reindirizza ai server Google, riceve il token di autorizzazione e, al primo accesso, obbliga il cittadino a completare il proprio profilo prima di emettere un JWT. Gli *Operatori* accedono tramite credenziali username/password; il sistema verifica le credenziali, confronta la password con l'hash bcrypt salvato e rilascia un JWT di sessione.

Interfaccia richiesta credenziali operatore:

Username e password dell'operatore, forniti tramite il form di login. La password viene confrontata con il digest bcrypt memorizzato nel database.

Interfaccia richiesta autorizzazione OAuth da Google:

Token OAuth 2.0 ottenuto al termine del flusso Google SSO. Il componente verifica l'autenticità del token, estrae l'e-mail e l'ID Google del cittadino e crea (o recupera) il profilo corrispondente.

Interfaccia fornita richiesta OAuth verso Google:

Reindirizzamento del browser del cittadino verso il server di autorizzazione Google (/auth/google), con i parametri `client_id`, `scope` e `redirect_uri`.

Interfaccia fornita JWT di sessione:

Token JWT firmato con il `JWT_SECRET`, contenente l'identificativo dell'utente e il suo ruolo (`operatore` o `cittadino`). Viene incluso nelle successive richieste come header `Authorization: Bearer <token>`.

Interfaccia richiesta completamento profilo cittadino:

Al primo accesso Google, se `profiloCompleto = false`, il cittadino deve fornire data di nascita, circoscrizione di residenza, categoria occupazionale e genere tramite `POST /auth/complete-profile`. Nome ed e-mail sono invece importati automaticamente dal profilo Google.

Interfaccia fornita autenticazione:

Permette all'utente autenticato di accedere a tutte le funzionalità a lui riservate, garantendo l'accesso sicuro alle risorse protette tramite il middleware `protect`.

CMP2: Gestione database**Descrizione:**

Il componente funge da strato di persistenza del sistema. Gestisce la connessione al database MongoDB e mette a disposizione degli altri componenti le operazioni CRUD sulle entità del dominio (*Cittadino*, *Operatore*, *Consultazione*, *Domanda*, *Iniziativa*, *RispostaConsultazione*, *VotoIniziativa*, *Categoria*, *Notifica*). Applica la validazione degli schemi Mongoose prima di ogni scrittura.

Interfaccia richiesta dati di registrazione operatore:

Username, password in chiaro, nome e cognome. Il componente salva la password come hash bcrypt e imposta `isRoot` a `false`: l'unico operatore *root* è quello creato dal seed iniziale.

Interfaccia richiesta dati profilo cittadino:

Dati del cittadino (data di nascita, circoscrizione, genere, categoria occupazionale) necessari per portare `profiloCompleto` a `true`; nome e cognome, importati da Google, sono aggiornabili separatamente.

Interfaccia richiesta dati consultazione:

Tipo (votazione | sondaggio), titolo, descrizione, date di inizio, fine e discussione, domande associate, stato iniziale (`bozza`).

Interfaccia richiesta risposta a consultazione:

Identificativo cittadino, identificativo consultazione, opzioni selezionate o testo libero. Viene verificato che il cittadino non abbia già risposto alla stessa consultazione.

Interfaccia richiesta dati iniziativa:

Titolo, descrizione, categoria, identificativo del cittadino proponente e stato iniziale (`in_attesa`).

Interfaccia fornita entità recuperate:

Oggetti JSON restituiti ai componenti che ne fanno richiesta (liste paginate, singoli documenti, aggregazioni statistiche).

Interfaccia fornita modifiche al database:

Conferma delle operazioni di creazione, aggiornamento o eliminazione dei documenti.

CMP3: Gestione consultazioni**Descrizione:**

Il componente si occupa del ciclo di vita delle consultazioni pubbliche, che comprendono sia le *votazioni* (con una singola domanda a risposta singola o multipla) sia i *sondaggi* (con più domande). Gli operatori creano le consultazioni in stato `bozza` e le pubblicano; una volta pubblicate diventano votabili a partire dalla `data_inizio`. Lo scheduler (*CMP7*) attiva automaticamente le bozze la cui `data_inizio` è trascorsa e porta a `concluso` le consultazioni alla `data_fine`. I cittadini possono rispondere alle consultazioni attive; l'operatore può infine archiviare quelle concluse.

Interfaccia richiesta autenticazione:

JWT valido per distinguere il ruolo: solo gli operatori possono creare/modificare consultazioni; i cittadini possono solo leggerle e rispondere.

Interfaccia richiesta dati nuova consultazione:

Tipo, titolo, date, domande (una per votazione, array per sondaggio) e lo stato iniziale.

Interfaccia richiesta risposta del cittadino:

Identificativo cittadino, identificativo consultazione e opzioni selezionate.

Interfaccia fornita lista consultazioni:

Elenco filtrato per tipo e/o stato, con paginazione e ordinamento per data.

Interfaccia fornita dettaglio consultazione:

Singolo documento con le domande associate e i metadati (stato, date, autore).

Interfaccia fornita conferma risposta registrata:

Esito positivo o negativo della registrazione della risposta del cittadino.

Interfaccia fornita statistiche votazione:

Aggregazione del numero di voti per opzione, con le relative percentuali, disponibile a qualunque utente autenticato. Le aggregazioni demografiche dei votanti sono invece riservate agli operatori (si veda *CMP6*).

CMP4: Gestione iniziative**Descrizione:**

Il componente gestisce le iniziative civiche proposte dai cittadini. Un cittadino può creare un'iniziativa fornendo titolo, descrizione e categoria; gli altri cittadini possono sostenerla con un voto. Gli operatori moderano le iniziative, portandole dallo stato `in_attesa` a `approvata` o `rifiutata` con una motivazione. Al termine della moderazione, il componente delega a *CMP8* la creazione della notifica per il cittadino proponente.

Interfaccia richiesta autenticazione:

JWT valido; il componente verifica il ruolo per decidere quali operazioni sono consentite.

Interfaccia richiesta dati nuova iniziativa:

Titolo, descrizione, identificativo categoria. L'identificativo del cittadino viene estratto dal JWT.

Interfaccia richiesta decisione di moderazione:

Nuovo stato (`approvata` | `rifiutata`) e motivazione testuale, forniti dall'operatore tramite `PATCH /iniziative/:id/modera`.

Interfaccia fornita lista iniziative:

Elenco delle iniziative, filtrabile per stato e categoria, con il numero di voti ricevuti.

Interfaccia fornita dettaglio iniziativa:

Documento con tutti i metadati e la descrizione completa, arricchito come la lista con il nome della categoria, il nominativo del cittadino proponente e il numero di voti ricevuti; per i cittadini include il flag `ha_votato`.

Interfaccia fornita esito voto iniziativa:

Conferma dell'aggiunta o rimozione del voto di supporto di un cittadino a un'iniziativa.

CMP5: Gestione profilo cittadino**Descrizione:**

Il componente raggruppa le funzionalità legate al profilo del cittadino: la visualizzazione dei propri dati, il completamento del profilo al primo accesso, la modifica dei dati anagrafici e il logout. Per gli operatori espone funzionalità analoghe: visualizzazione del profilo, cambio password e promozione a *root*.

Interfaccia richiesta autenticazione:

JWT valido per garantire che ogni utente acceda solo al proprio profilo.

Interfaccia richiesta dati completamento profilo:

Data di nascita, circoscrizione, categoria occupazionale e genere forniti dal cittadino tramite POST `/auth/complete-profile`.

Interfaccia richiesta dati aggiornamento anagrafica:

Nome e cognome aggiornati, inviati dal cittadino tramite PATCH `/cittadino/me`.

Interfaccia richiesta nuova password operatore:

Password attuale e nuova password, utilizzate per il cambio credenziali (PATCH `/operatore/me/password`).

Interfaccia fornita profilo utente:

Dati anagrafici e metadati dell'account (data di registrazione, stato `loggedIn`, `profiloCompleto`).

Interfaccia fornita conferma logout:

Aggiornamento del flag `loggedIn` a `false` e risposta di conferma al client.

CMP6: Gestione statistiche**Descrizione:**

Il componente aggrega i dati di partecipazione alle consultazioni per produrre statistiche visualizzabili dagli operatori. Il *riepilogo sintetico* calcola, per ogni opzione, il numero di voti ricevuti e la percentuale sul totale dei votanti. Il *riepilogo demografico* incrocia i voti con i dati del cittadino votante e produce la distribuzione per genere, per fascia d'età (derivata dalla `dataNascita`: 18–25, 26–35, 36–50, oltre 50), per categoria occupazionale e l'andamento giornaliero della partecipazione. Espone queste aggregazioni tramite le API di reportistica.

Interfaccia richiesta dati risposte:

Insieme delle *RispostaConsultazione* associate a una consultazione, con gli identificativi dei cittadini rispondenti.

Interfaccia richiesta dati anagrafici cittadini:

Dati dei cittadini, recuperati tramite *join* sulla collezione *Cittadino* nel momento in cui la statistica viene richiesta.

Interfaccia fornita statistiche aggregate:

JSON con il conteggio voti per opzione e le distribuzioni per aggregazioni.

CMP7: Scheduler consultazioni**Descrizione:**

Il componente implementa un job schedulato (via `node-cron`, esecuzione oraria) che gestisce le transizioni automatiche di stato delle consultazioni. All'avvio dell'applicazione esegue immediatamente un ciclo di controllo. Le transizioni gestite sono: `bozza` → `attivo` quando `data_inizio` è trascorsa e `attivo` → `concluso` quando `data_fine` è trascorsa. Le sole transizioni non automatiche sono l'archiviazione (`concluso` → `archiviato`) e la pubblicazione anticipata di una bozza, entrambe a carico dell'operatore.

Interfaccia richiesta lista consultazioni attive/in bozza:

Interrogazione periodica al database per recuperare le consultazioni in stato `bozza` o `attivo` che soddisfano i criteri di transizione.

Interfaccia fornita aggiornamento stato consultazione:

Scrittura del nuovo stato nel database.

CMP8: Sistema di notifiche**Descrizione:**

Il componente gestisce le notifiche in-app destinate ai cittadini. Attualmente le notifiche vengono generate automaticamente dal componente *CMP4 Gestione Iniziative* al termine della moderazione: il cittadino proponente riceve una notifica quando la propria iniziativa viene `approvata` o `rifiutata`. Il componente espone API per recuperare le notifiche, segnarle come lette singolarmente o tutte insieme. Il frontend effettua polling ogni 2 minuti per aggiornare il contatore delle notifiche non lette nella topbar.

Interfaccia richiesta evento di moderazione:

Segnale proveniente da *CMP4* al termine della `PATCH /iniziative/:id/modera:` contiene lo stato assegnato (`approvata` | `rifiutata`), la motivazione e l'identificativo del cittadino destinatario.

Interfaccia richiesta autenticazione:

JWT valido con ruolo `cittadino`; ogni cittadino può leggere e gestire solo le proprie notifiche.

Interfaccia fornita lista notifiche:

Elenco delle notifiche del cittadino autenticato, ordinate dalla più recente, con il campo `letta` per distinguere le notifiche nuove da quelle già viste (`GET /notifiche`).

Interfaccia fornita conferma lettura:

Aggiornamento del flag `letta` a `true` per una singola notifica (`PATCH /notifiche/:id/letta`) o per tutte quelle non lette (`PATCH /notifiche/leggi-tutte`).

1.2. Diagramma dei componenti

Viene riportato di seguito il diagramma complessivo di tutti i componenti di sistema e le loro interconnessioni descritte in precedenza.

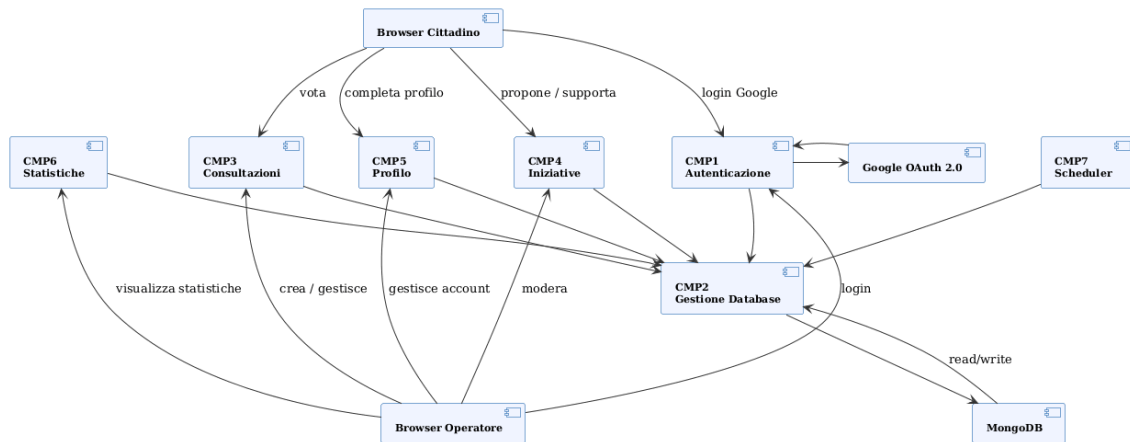


Figura 1: Diagramma dei componenti IoSonoTrento

2. Diagramma delle classi

Nel presente capitolo vengono presentate le classi previste nell'ambito del progetto **IoSonoTrento**. Ogni componente presentato nel capitolo precedente si riflette in una o più classi del dominio. Tutte le classi sono caratterizzate da: un nome, una lista di attributi che identificano i dati gestiti, e una lista di metodi che definiscono le operazioni previste. In questo processo si è proceduto a massimizzare la coesione e minimizzare l'accoppiamento tra classi.

1. Cittadino

Il *Cittadino* è l'utente finale della piattaforma. Accede tramite Google OAuth, può partecipare a consultazioni, proporre e votare iniziative. Al primo accesso il suo profilo è incompleto (`profiloCompleto = false`) e deve essere completato prima di poter operare.

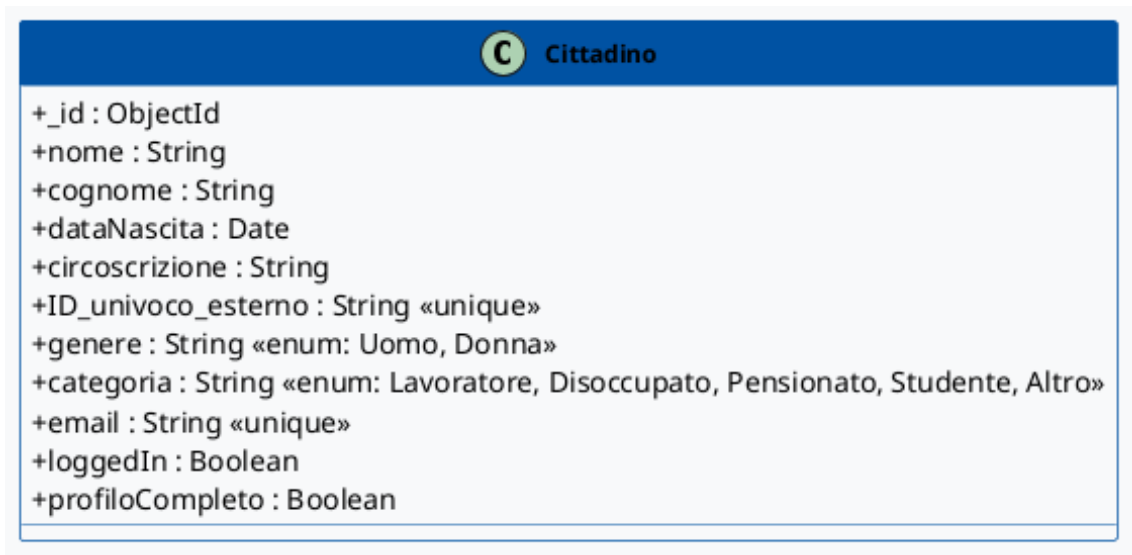


Figura 2: Classe Cittadino

Attributi principali:

- `_id`: ObjectId univoco MongoDB
- `nome`, `cognome`: dati anagrafici
- `email`: indirizzo e-mail Google
- `ID_univoco_esterno`: Google ID (sub claim del token OAuth)
- `dataNascita`: data di nascita (usata per le statistiche per fascia d'età)
- `circoscrizione`: circoscrizione di residenza
- `genere`: Uomo | Donna (usato per le statistiche demografiche)
- `categoria`: categoria occupazionale (Lavoratore, Disoccupato, Pensionato, Studente, Altro)
- `profiloCompleto`: booleano, `false` al primo accesso
- `loggedIn`: booleano, aggiornato a ogni login/logout

Metodi principali:

- `completeProfile(dati)`: imposta `dataNascita`, `circoscrizione`, `genere` e `categoria`; porta `profiloCompleto` a `true`
- `updateProfile(dati)`: aggiorna `nome` e `cognome`
- `logout()`: aggiorna `loggedIn` a `false`
- `getProfile()`: restituisce i dati del profilo

2. Operatore

L'*Operatore* è il dipendente comunale che gestisce la piattaforma: crea consultazioni, modera le iniziative, gestisce gli altri operatori. L'operatore *root* (creato dal seed iniziale) può promuovere altri operatori.

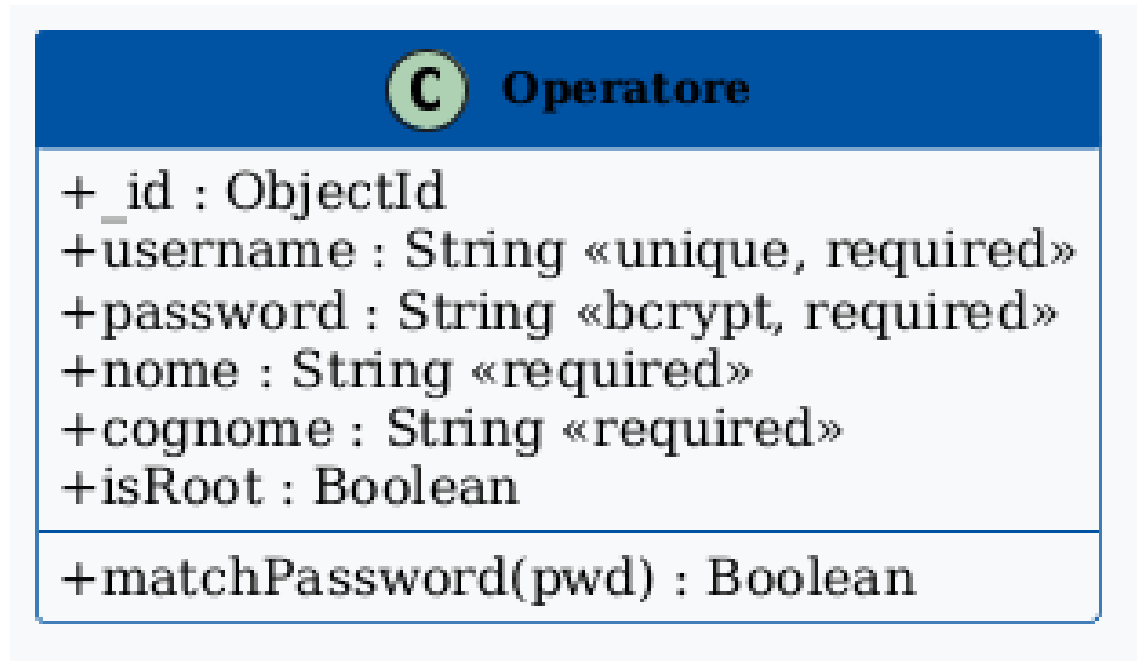


Figura 3: Classe Operatore

Attributi principali:

- `_id`: ObjectId univoco MongoDB
- `username`: identificativo di accesso univoco
- `password`: hash bcrypt della password
- `nome`, `cognome`: dati anagrafici del dipendente comunale
- `isRoot`: booleano, consente la gestione degli altri operatori

Metodi principali:

- `login(username, password)`: verifica le credenziali e rilascia un JWT
- `changePassword(vecchiaPassword, nuovaPassword)`: aggiorna l'hash bcrypt
- `promote(operatoreId)`: promuove un operatore (solo per `isRoot`)
- `register(username, password, nome, cognome)`: crea un nuovo account operatore (solo per `isRoot`); il nuovo account ha sempre `isRoot = false`
- `getProfile()`: restituisce le informazioni dell'operatore.

3. Consultazione

La *Consultazione* rappresenta il fulcro della partecipazione civica. Utilizza un pattern discriminato di Mongoose: se il tipo è *votazione*, ha una singola domanda (*ID_domanda*); se è *sondaggio*, ha un array di domande (*ID_domande[]*). Questi due campi sono mutuamente esclusivi. Lo stato *concluso* viene impostato automaticamente dal sistema allo scadere di *data_fine*, senza intervento dell'operatore.

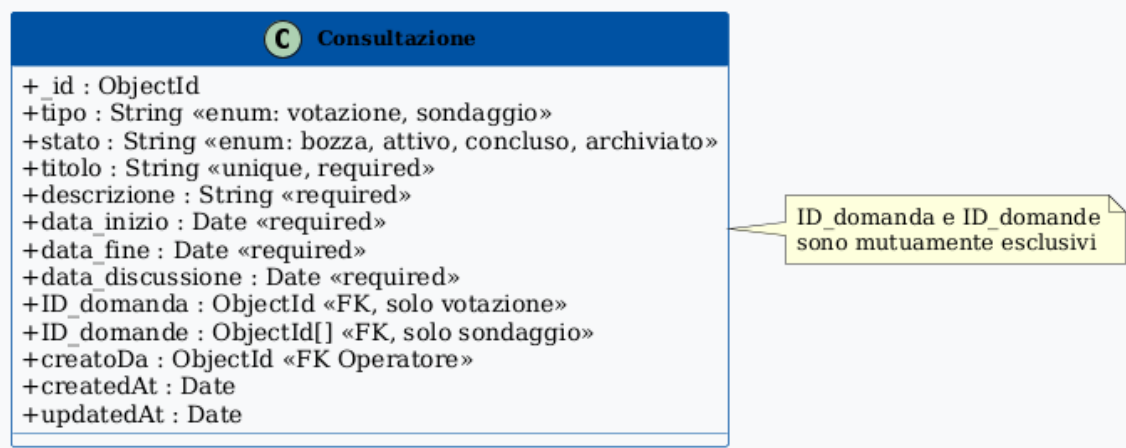


Figura 4: Classe Consultazione

Attributi principali:

- tipo: votazione | sondaggio
- titolo: titolo della consultazione (univoco)
- descrizione: testo descrittivo della consultazione
- stato: bozza → attivo → concluso → archiviato
- data_inizio, data_fine: intervallo temporale di attivazione
- data_discussione: data della sessione pubblica di discussione
- ID_domanda: riferimento alla domanda (solo votazione)
- ID_domande: array di riferimenti alle domande (solo sondaggio)
- creatoDa: riferimento all'operatore creatore

Metodi principali:

- update(modifiche): se in stato di bozza, permette di modificarne i campi ammessi.
- publish(): porta lo stato da bozza ad attivo
- conclude(): porta lo stato da attivo a concluso
- archive(): porta lo stato da concluso ad archiviato
- getStatistiche(): aggrega e restituisce le statistiche di partecipazione

4. Domanda

La *Domanda* è l'unità di quesito associabile a una consultazione. Supporta due modalità: risposta singola (radio) e risposta multipla (checkbox).

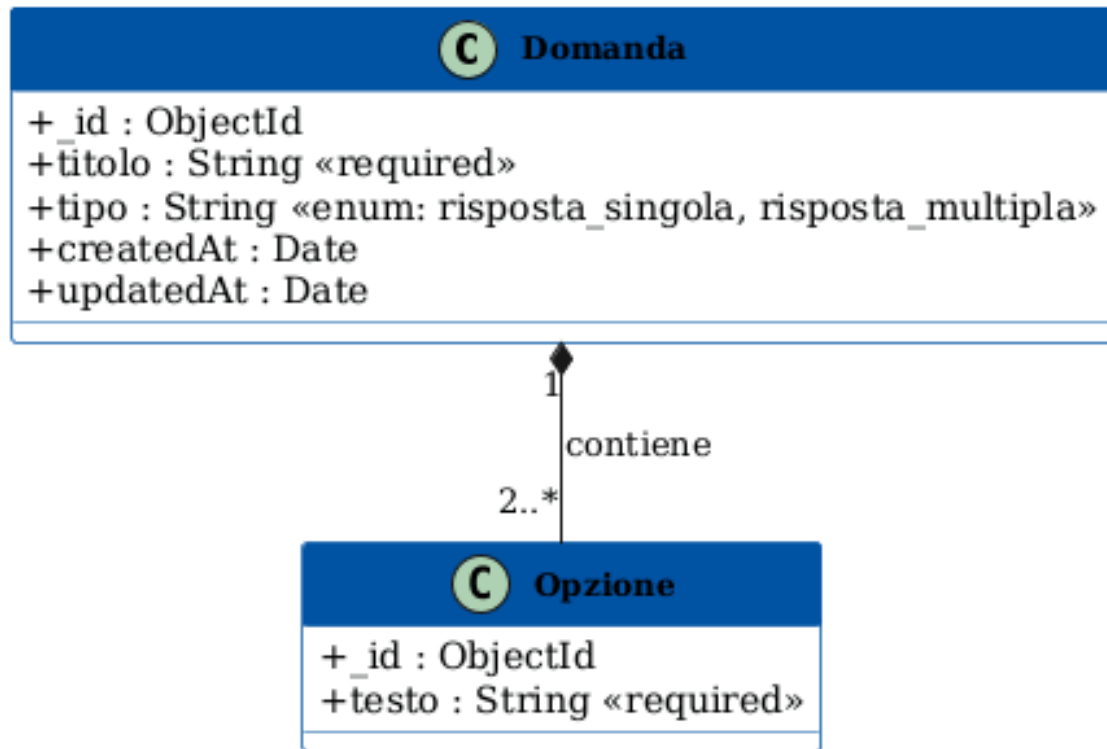


Figura 5: Classe Domanda

Attributi principali:

- **titolo**: testo del quesito
- **opzioni**: array di sotto-documenti *Opzione*, ciascuno con un proprio `_id` e il campo `testo`; ne servono almeno due. È l'`_id` dell'opzione a essere referenziato dalle risposte dei cittadini
- **tipo**: `risposta_singola` | `risposta_multipla`

5. Iniziativa

L'*Iniziativa* è la proposta civica che un cittadino può presentare alla municipalità. Dopo la creazione è in attesa di moderazione da parte di un operatore.

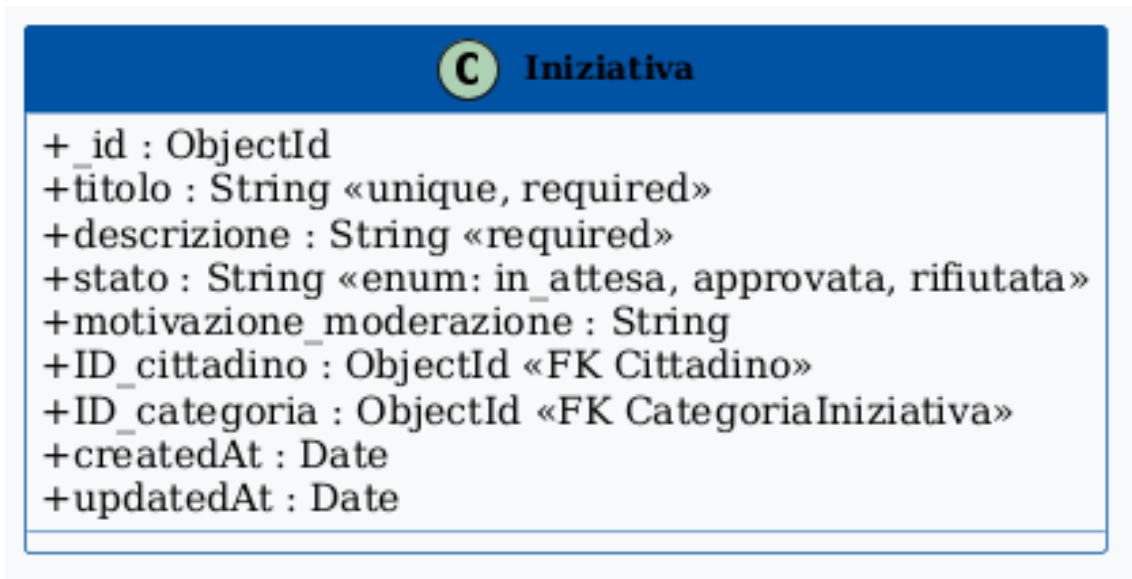


Figura 6: Classe Iniziativa

Attributi principali:

- titolo: titolo dell'iniziativa
- descrizione: testo esteso della proposta
- ID_cittadino: riferimento al cittadino proponente
- ID_categoria: riferimento alla categoria tematica
- stato: in_attesa | approvata | rifiutata
- motivazione_moderazione: testo della motivazione dell'operatore

Metodi principali:

- modera(stato, motivazione): aggiorna lo stato e salva la motivazione
- getVoti(): restituisce il numero di voti di supporto ricevuti

6. RispostaConsultazione

La *RispostaConsultazione* è la classe di join tra *Cittadino* e *Consultazione*. Registra le opzioni selezionate o le risposte testuali per ogni domanda a cui il cittadino ha risposto.

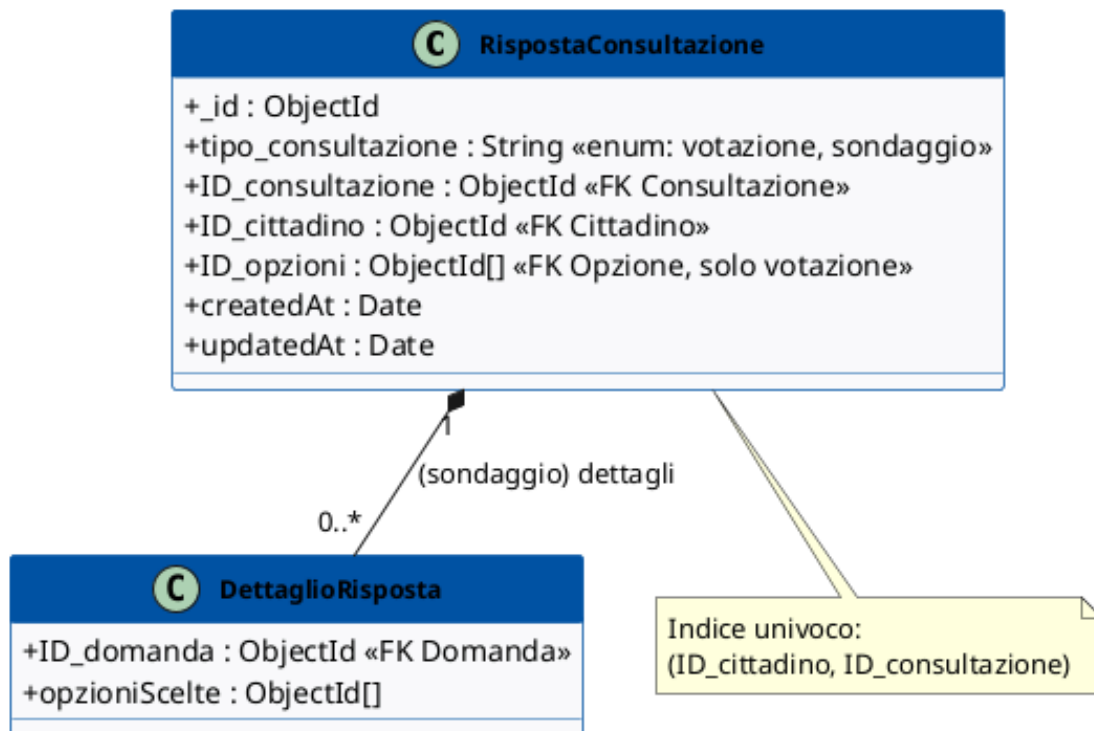


Figura 7: Classe Risposta Consultazione

Attributi principali:

- `ID_cittadino`: riferimento al cittadino rispondente
- `ID_consultazione`: riferimento alla consultazione
- `tipo_consultazione`: `votazione` | `sondaggio`; determina quale dei due campi seguenti è valorizzato
- `ID_opzioni`: array degli `_id` delle opzioni scelte (solo votazione: un elemento per la risposta singola, più di uno per quella multipla)
- `dettagliRisposte`: array di coppie `ID_domanda → opzioniScelte[]` (solo sondaggio)
- `createdAt`, `updatedAt`: timestamp gestiti da Mongoose

Un indice univoco sulla coppia (`ID_cittadino`, `ID_consultazione`) garantisce a livello di database che un cittadino risponda una sola volta a una consultazione.

Metodi principali:

- `esiste(cittadinoId, consultazioneId)`: controlla un eventuale risposta già presente

7. VotoIniziativa

La *VotoIniziativa* è la classe di join tra *Cittadino* e *Iniziativa*. Registra il supporto espresso da un cittadino a un'iniziativa. Un cittadino può togliere il proprio voto (DELETE /cittadino/vote/iniziativa/:id).

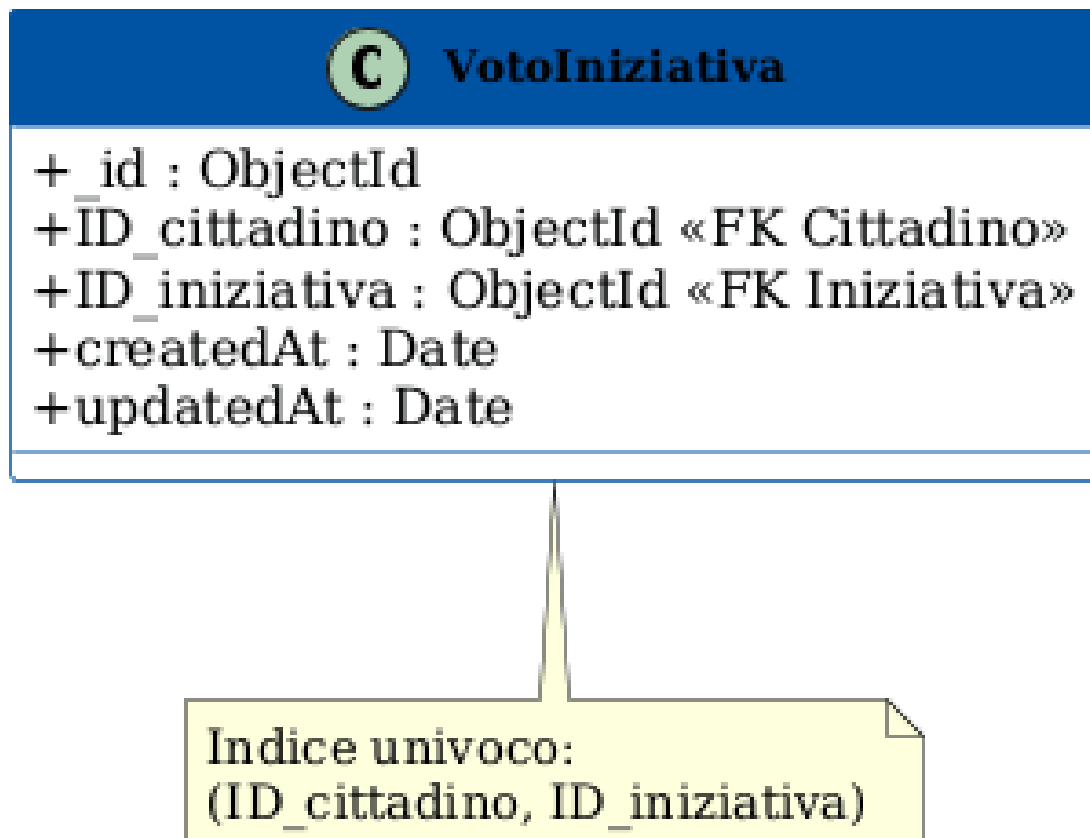


Figura 8: Classe Votto Iniziativa

Attributi principali:

- `ID_cittadino`: riferimento al cittadino votante
- `ID_iniziativa`: riferimento all'iniziativa supportata
- `createdAt`, `updatedAt`: timestamp gestiti da Mongoose

Un indice univoco sulla coppia (`ID_cittadino`, `ID_iniziativa`) impedisce a un cittadino di votare due volte la stessa iniziativa.

Metodi principali:

- `esiste(cittadinoId,iniziativaId)`: controlla un eventuale risposta già presente

8. Categoria Iniziativa

La *Categoria Iniziativa* è una tassonomia tematica usata per classificare le iniziative (es. *Mobilità, Ambiente, Cultura*).

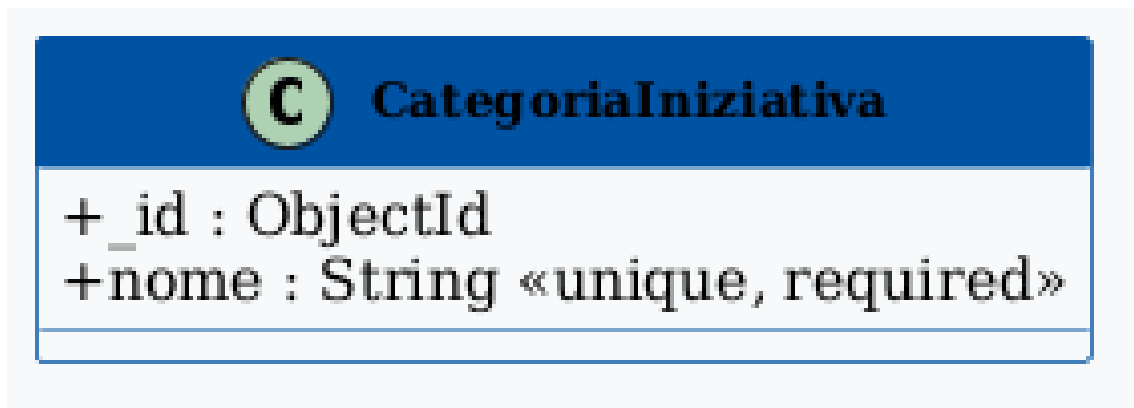


Figura 9: Classe Categoria Iniziativa

Attributi principali:

- **nome**: nome della categoria (univoco)

9. Notifica

La *Notifica* rappresenta un messaggio in-app generato automaticamente dal sistema e destinato a un cittadino. Attualmente viene creata alla moderazione di un'iniziativa per informare il proponente dell'esito (approvata o rifiutata).

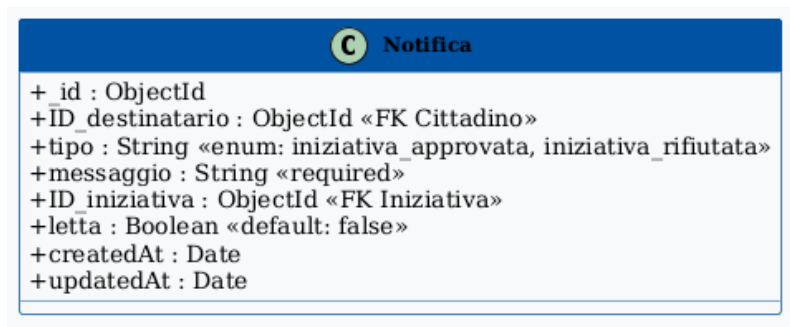


Figura 10: Classe Notifica

Attributi principali:

- **ID_destinatario**: riferimento al cittadino destinatario
- **tipo**: `iniziativa_approvata` | `iniziativa_rifiutata`
- **messaggio**: testo della notifica generato automaticamente
- **ID_iniziativa**: riferimento all'iniziativa a cui si riferisce la notifica

- **letta**: booleano, **false** alla creazione; portato a **true** quando il cittadino apre il pannello notifiche

Metodi principali:

- `marcaLetta()`: imposta letta a true
- `marcaTutteLette()`: marca come lette tutte le notifiche del destinatario che invoca il metodo.

2.1. Diagramma delle classi complessivo

Viene riportato di seguito il diagramma delle classi complessivo che mostra le relazioni tra tutte le classi del dominio.

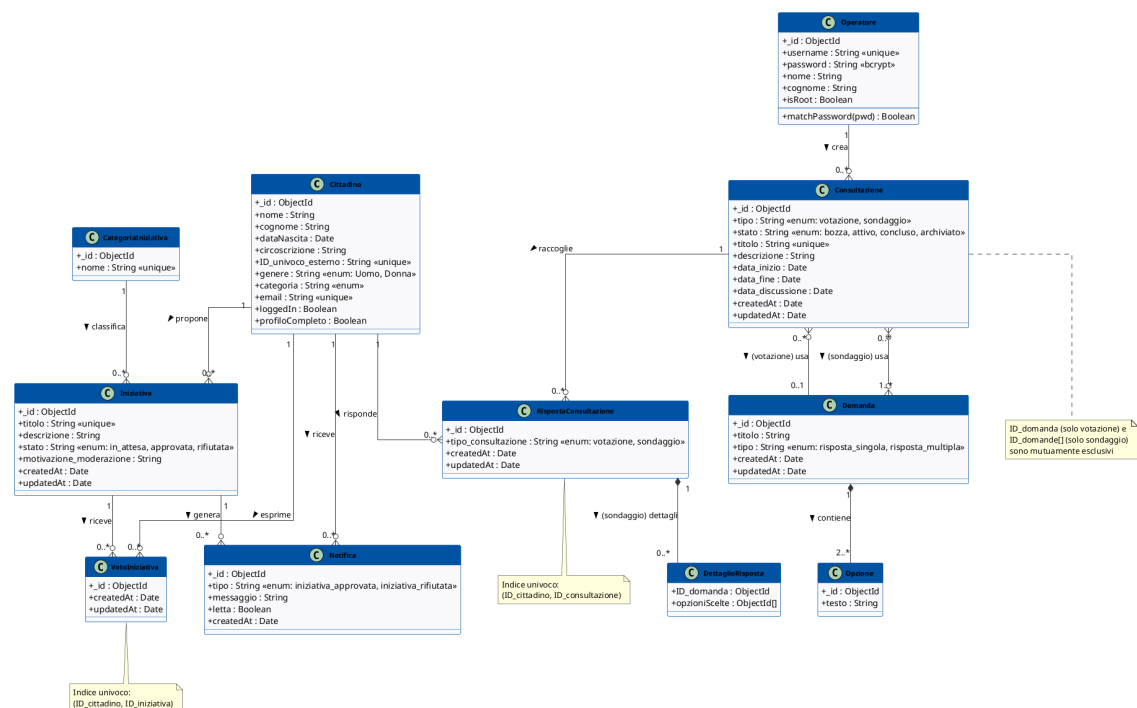


Figura 11: Diagramma delle classi complessivo IoSonoTrento

3. Dal class diagram alle API

Il diagramma delle classi delineato nella sezione precedente si traduce direttamente nelle API REST esposte dal backend Node.js / Express. Ogni metodo delle classi corrisponde a uno o più endpoint. Di seguito viene riportata la mappatura tra classi/metodi e route HTTP.

Classe / Metodo	Metodo HTTP	Endpoint
<code>Operatore.login()</code>	POST	<code>/operatore/login</code>

Classe / Metodo	Metodo HTTP	Endpoint
Operatore.register()	POST	/operatore/register *
Operatore.getProfile()	GET	/operatore/profile
Operatore.changePassword()	PATCH	/operatore/me/password
Operatore.promote()	PATCH	/operatore/:operatorId/promote *
Cittadino.loginGoogle()	GET	/auth/google
Cittadino.oauthCallback()	GET	/auth/google/callback
Cittadino.completeProfile()	POST	/auth/complete-profile
Cittadino.getProfile()	GET	/cittadino/profile
Cittadino.updateProfile()	PATCH	/cittadino/me
Cittadino.logout()	POST	/cittadino/logout
Consultazione.create() (votazione)	POST	/votazioni
Consultazione.list() (votazione, op.)	GET	/votazioni
Consultazione.list() (votazione, cit.)	GET	/votazioni/cittadino §
Consultazione.getById() (votazione)	GET	/votazioni/:id †
Consultazione.update() (votazione)	PATCH	/votazioni/:id
Consultazione.delete() (votazione)	DELETE	/votazioni/:id
Consultazione.publish() (votazione)	PATCH	/votazioni/:id/publish
Consultazione.archive() (votazione)	PATCH	/votazioni/:id/archive
Consultazione.getStatistiche()	GET	/votazioni/:id/riepilogo
Consultazione.getStatistiche() (demogr.)	GET	/votazioni/:id/riepilogo/demografico
Consultazione.conclude()	—	(automatico alla scadenza di data_fine)
RispostaConsultazione.create()	POST	/cittadino/vote/votazione ‡
Consultazione.create() (sondaggio)	POST	/sondaggio
Consultazione.list() (sondaggio, op.)	GET	/sondaggio
Consultazione.list() (sondaggio, cit.)	GET	/sondaggio/cittadino §
Consultazione.getById() (sondaggio)	GET	/sondaggio/:id
Consultazione.update() (sondaggio)	PATCH	/sondaggio/:id
Consultazione.delete() (sondaggio)	DELETE	/sondaggio/:id

Classe / Metodo	Metodo HTTP	Endpoint
Consultazione.publish() (sondaggio)	PATCH	/sondaggio/:id/publish
Consultazione.archive() (sondaggio)	PATCH	/sondaggio/:id/archive
Consultazione.getStatistiche()	GET	/sondaggio/:id/riepilogo
Consultazione.getStatistiche() (filtri)	POST	/sondaggio/:id/riepilogo/agggregato [†]
Consultazione.conclude()	—	(automatico alla scadenza di data_fine)
RispostaConsultazione.create()	POST	/cittadino/vote/sondaggio
Iniziativa.create()	POST	/iniziative
Iniziativa.list()	GET	/iniziative
Iniziativa.ricerca()	POST	/iniziative/ricerca
Iniziativa.getId()	GET	/iniziative/:id
Iniziativa.update()	PATCH	/iniziative/:id
Iniziativa.delete()	DELETE	/iniziative/:id
Iniziativa.modera()	PATCH	/iniziative/:id/modera
VotoIniziativa.create()	POST	/cittadino/vote/iniziativa
VotoIniziativa.delete()	DELETE	/cittadino/vote/iniziativa/:id
Categoria.create()	POST	/categorie
Categoria.list()	GET	/categorie
Categoria.getId()	GET	/categorie/:id
Categoria.update()	PATCH	/categorie/:id
Categoria.delete()	DELETE	/categorie/:id
Notifica.list()	GET	/notifiche
Notifica.marcaLetta()	PATCH	/notifiche/:id/letta
Notifica.marcaTutteLette()	PATCH	/notifiche/leggi-tutte

[†] GET /votazioni/:id è accessibile sia all'operatore che al cittadino. L'operatore riceve la votazione in qualsiasi stato; il cittadino vede solo votazioni in stato **attivo** o **concluso** e la risposta include il campo booleano `voted` che indica se il cittadino ha già espresso il proprio voto.

[‡] POST /cittadino/vote/votazione accetta in `opzioniId` una o più opzioni: per le domande a risposta singola ne è ammessa esattamente una, per quelle a risposta multipla anche più di una. Verifica che la votazione sia in stato **attivo** prima di accettare il voto e che le opzioni appartengano alla domanda; restituisce 403 se lo stato non è **attivo** o se il cittadino ha già votato.

[§] GET /votazioni/cittadino e GET /sondaggio/cittadino restituiscono solo consultazioni in stato **attivo** o **concluso**. Ogni elemento include il campo booleano `voted` che indica se il cittadino autenticato ha già partecipato a quella consultazione (calcolato con una singola query batch su `RispostaConsultazione`).

* `POST /operatore/register` e `PATCH /operatore/:operatoreId/promote` sono riservati agli operatori con flag `isRoot`; gli altri operatori ricevono 403.

¶ `POST /sondaggio/:id/riepilogo/aggregato` è riservato all'operatore e accetta nel corpo della richiesta i filtri demografici da applicare all'aggregazione; per questo motivo usa il metodo `POST` anziché `GET`. Analogamente, `GET /votazioni/:id/riepilogo/demografici` è riservato all'operatore, mentre i due riepiloghi sintetici (`/votazioni/:id/riepilogo` e `/sondaggio/:id/riepilogo`) sono accessibili a qualunque utente autenticato.

La protezione degli endpoint è garantita dal middleware `protect` (verifica JWT) e da `restrictTo(['ruolo'])` che limita l'accesso in base al ruolo. Il middleware `requireProfiloCompleto` blocca le operazioni di voto e la creazione di iniziative finché il cittadino non ha completato il proprio profilo, mentre `validateObjectId` previene `CastError` di Mongoose sui parametri `/:id`. Le route `POST /operatore/login` e `POST /auth/complete-profile` sono inoltre protette da rate limiting (`express-rate-limit`). La documentazione completa degli endpoint è disponibile tramite Swagger UI all'indirizzo `http://localhost:8000/docs`.